SANCTIWALK: A MOBILE APP TOUR GUIDE SYSTEM FOR MARY HELP OF CHRISTIANS PARISH AND SAN ROQUE CATHEDRAL PARISH, DIOCESE OF KALOOKAN

Appendix: Source Code

Generated from the project source on 2026-10-24

About This Appendix

This appendix presents the source code that implements each major feature of SanctiWalk. Every line shown is the actual code from the project.

Most listings are extracts. Where a listing is an extract, its heading gives the line numbers it occupies in the original file, so the full context can be found directly.

Long explanatory comment blocks have been collapsed to their opening sentence so that the code itself remains readable in print. No comment has been rewritten: the sentence shown is the one written in the file.

The complete source, with every file and every comment intact, runs to approximately 1,300 pages and is submitted digitally as SanctiWalk_Complete_Source_Code_Appendix.pdf. The project repository is https://github.com/Gifeys/SanctDemo.

Table of Contents

About This Appendix 2
Table of Contents 3
1. Mobile Application — Shell and Navigation 5
SRC-01 main.tsx 5
SRC-02 RootGate.tsx 7
2. Mass Schedule and Reminders 8
SRC-03 sundayMasses.ts 8
SRC-04 schedule.ts 9
SRC-05 massScheduleWatch.ts 10
SRC-06 openingSubscriptions.ts 12
3. Prayer, Narration and Speech 13
SRC-07 speech.ts 13
SRC-08 DailyRosary.tsx 15
4. Artificial Intelligence — Scanner and Parish Assistant 17
SRC-09 sanctiGrounding.ts 17
SRC-10 askSancti.ts 19
SRC-11 scanQuota.ts 20
5. Maps and Navigation 21
SRC-12 followCamera.ts 21
SRC-13 geo.ts 22
6. Backend and API 23
SRC-14 server.ts 23
SRC-15 server.ts 25
SRC-16 server.ts 30
7. Firebase, Database and Security 32
SRC-17 firebase.ts 32
SRC-18 authFlow.ts 33

SRC-19 availability.ts 34
SRC-20 firestore.rules 35
8. Administrative Portal 40
SRC-21 AdminItems.tsx 40
SRC-22 pageEvents.ts 43
9. Map Placer and Station Pipeline 45
SRC-23 deploy-stations.py 45
SRC-24 save_server.py 47
10. Unity Augmented Reality 49
SRC-25 MarkerScan.cs 49
SRC-26 ChurchOriginTest.cs 51
SRC-27 MuseumStation.cs 55
SRC-28 ArFlowUI.cs 58
SRC-29 MultiSetVps.cs 60
11. Build, Packaging and Deployment 61
SRC-30 capacitor.config.ts 61
SRC-31 AndroidManifest.xml 62
SRC-32 vercel.json 64
Index of Listings 65

If blank, press Ctrl+A then F9 in Word.

1. Mobile Application — Shell and Navigation

SRC-01 main.tsx

File: SanctDemo/src/main.tsx • whole file • 120 lines in full

What this code does: Every route in the application, including the relocated administration path and the landing page.

```
import {StrictMode} from 'react';
import {createRoot} from 'react-dom/client';
import { SpotlightProvider } from './components/Spotlight';
import {BrowserRouter, Routes, Route, Navigate} from 'react-router-dom';
import App from './App.tsx';
import FinishSignIn from './routes/FinishSignIn.tsx';
import SignUpFlow from './routes/SignUpFlow.tsx';
import AdminApp from './admin/AdminApp.tsx';
import RootGate from './components/RootGate.tsx';
import Landing from './components/Landing.tsx';
import './index.css';
import { registerServiceWorker } from './lib/registerServiceWorker';

// Uncaught errors, with their stack, into the Android log.
window.addEventListener('error', event => {
  const err = event.error as Error | undefined;
  console.error(
    '[SW-ERROR]', event.message,
    '| at', event.filename || '(no file)', event.lineno + ':' + event.colno,
    '| stack:', err?.stack ?? '(none)',
  );
});

window.addEventListener('unhandledrejection', event => {
  const reason = event.reason as { message?: string; stack?: string } | undefined;
  console.error(
    '[SW-REJECT]', reason?.message ?? String(event.reason),
    '| stack:', reason?.stack ?? '(none)',
  );
});

// Routing was added AROUND the existing app, not through it.
createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <BrowserRouter>
      <Routes>
        <Route path="/signup" element={<SignUpFlow />} />
        <Route path="/finish-sign-in" element={<FinishSignIn />} />
        {/* The parish office. Not at /admin.

            Moving it is worth almost nothing against anyone who looks -
            the path is in the JavaScript bundle, which is public - and
            it is NOT what keeps the office safe. That is Firebase Auth,
            the admins collection, and the Firestore rules Google
            enforces on their own servers. What it does buy is quiet:
            the bots that sweep the internet for /admin and /wp-admin
            all day stop finding a login form here.

            /admin itself now answers like any other unknown path, so
            the move cannot be undone by guessing the old one. */}
        <Route path="/parish-office-gav8thhe/*" element={<AdminApp />} />
```

```
        {/* Both earlier addresses are dead ends now, so neither the
            original guess nor the first move reopens the office. */}
        <Route path="/admin/*" element={<Navigate to="/" replace />} />
        <Route path="/parish-office/*" element={<Navigate to="/" replace />} />

        {/* The root is two different things.
            A phone gets the app, because somebody on a phone came to use
            it. A laptop gets the page explaining it, because somebody on
            a laptop is deciding whether to look. RootGate renders rather
            than redirects, so the address stays "/" - which matters,
            because that is the PWA's start_url and the service worker's
            cached shell. */}
        <Route path="/" element={<RootGate />} />

        {/* The app, unconditionally. Where "Explore SanctiWalk" goes, and
            the way a laptop reaches the app at all. */}
        <Route path="/app" element={<SpotlightProvider><App /></SpotlightProvider>} />

        {/* The promotional page, unconditionally - so it can be linked to
            from a poster or a post and still be readable on a phone. */}
        <Route path="/welcome" element={<Landing />} />

        {/* Everything else is the app: a catch-all rather than an exact "/"
            so any deep link the pilgrim app already understands still
            reaches it. The printed station posters encode /s/<CODE>, and
            they land here. */}
        {/* The spotlight lives inside the pilgrim app's route only. The
            parish office has no tutorial and no assistant, and a
            provider wrapping both would put a portal on the admin's
            document for no reason. */}
        <Route path="*" element={<SpotlightProvider><App /></SpotlightProvider>} />
      </Routes>
    </BrowserRouter>
  </StrictMode>,
);

// Makes the web build installable on a home screen, and openable
// inside a church with no signal. A no-op in the APK and in dev.
registerServiceWorker();
```

SRC-02 RootGate.tsx

File: SanctDemo/src/components/RootGate.tsx • extract • 70 lines in full

What this code does: How the same build shows a marketing page in a browser and the app itself once installed.

Lines 35–53 of RootGate.tsx

```
function isInstalledApp(): boolean {
  if (typeof window === "undefined") return false;

  // The APK. Asked first because it is the only one of the three that
  // is a fact rather than an inference.
  try {
    if (Capacitor.isNativePlatform()) return true;
  } catch {
    // No Capacitor bridge: an ordinary browser. Fall through.
  }

  try {
    // iOS Safari predates the standard and uses its own flag.
    const iosStandalone = (window.navigator as { standalone?: boolean }).standalone === true;
    return iosStandalone || window.matchMedia?.("(display-mode: standalone)")?.matches === true;
  } catch {
    return false;
  }
}
```

2. Mass Schedule and Reminders

SRC-03 sundayMasses.ts

File: SanctDemo/src/lib/sundayMasses.ts • extract • 199 lines in full

What this code does: Builds the Mass card from the parish's own schedule, and the full week behind it.

Lines 87–90 of sundayMasses.ts

```
export function sundayMassCard(
  schedule: MassScheduleEntry[],
  now: Date,
  { verified = true }: { verified?: boolean } = {},
```

Lines 171–199 of sundayMasses.ts

```
export function weekAhead(schedule: MassScheduleEntry[], now: Date): MassDay[] {
  const start = new Date(now)
  start.setHours(0, 0, 0, 0)

  const days: MassDay[] = []

  for (let offset = 0; offset < 7; offset++) {
    const date = new Date(start)
    date.setDate(date.getDate() + offset)

    const name = DAY_NAMES[date.getDay()]
    const entry = schedule.find(e => e.day === name)

    days.push({
      day: name,
      date,
      times: entry
        ? parseTimes(entry.time).map(time => ({
            time,
            available: isTimeAvailable(entry, time),
          }))
        : [],
      ...(entry?.note ? { note: entry.note } : {}),
      isToday: offset === 0,
    })
  }

  return days
}
```

SRC-04 schedule.ts

File: SanctDemo/src/lib/schedule.ts • extract • 177 lines in full

What this code does: How a single Mass is suspended without removing it from the schedule.

Lines 26–28 of schedule.ts

```
export function isTimeAvailable(entry: MassScheduleEntry, time: string): boolean {
  return !(entry.unavailableTimes ?? []).includes(time.trim())
}
```

Lines 36–49 of schedule.ts

```
export function withTimeAvailability(
  entry: MassScheduleEntry,
  time: string,
  available: boolean,
): MassScheduleEntry {
  const next = new Set(entry.unavailableTimes ?? [])
  if (available) next.delete(time.trim())
  else next.add(time.trim())
  const list = [...next]
  // Dropped entirely when empty rather than written as [], so a parish
  // that never suspends anything keeps the document it always had.
  const { unavailableTimes: _old, ...rest } = entry
  return list.length > 0 ? { ...rest, unavailableTimes: list } : rest
}
```

SRC-05 massScheduleWatch.ts

File: SanctDemo/src/lib/massScheduleWatch.ts • extract • 243 lines in full

What this code does: Detects what the parish changed and words the notification.

Lines 79–96 of massScheduleWatch.ts

```
export function massFingerprint(schedule: MassEntryLike[] | null | undefined): string | null {
  if (!Array.isArray(schedule) || schedule.length === 0) return null
  const parts = schedule.map(e => {
    const day = (e.day ?? '').trim().toLowerCase()
    const times = partsOf(e.time).join(',')
    // Suspensions belong in the fingerprint.
    const off = (e.unavailableTimes ?? [])
      .map(t => t.trim().toLowerCase())
      .filter(Boolean)
      .sort()
      .join(',')
    return `${day}|${times}|${off}`
  })
  return parts.join(';')
}
```

Lines 111–140 of massScheduleWatch.ts

```
export function massDiff(
  previous: string | null,
  current: string | null,
): { suspended: MassMoved[]; restored: MassMoved[] } {
  const none = { suspended: [], restored: [] }
  if (!previous || !current || previous === 'UNAVAILABLE') return none

  const read = (fp: string): Map<string, Set<string>> => {
    const map = new Map<string, Set<string>>()
    for (const part of fp.split(';')) {
      const [day = '', , off = ''] = part.split('|')
      map.set(day, new Set(off.split(',').filter(Boolean)))
    }
    return map
  }

  const before = read(previous)
  const after = read(current)

  const suspended: MassMoved[] = []
  const restored: MassMoved[] = []

  for (const [day, nowOff] of after) {
    const wasOff = before.get(day) ?? new Set<string>()
    for (const time of nowOff) if (!wasOff.has(time)) suspended.push({ day, time })
    for (const time of wasOff) if (!nowOff.has(time)) restored.push({ day, time })
  }

  return { suspended, restored }
}
```

Lines 178–191 of massScheduleWatch.ts

```
export function massNotice(
  change: MassChange,
  parishName: string,
  // What exactly moved, from massDiff.
  moved?: { suspended: MassMoved[]; restored: MassMoved[] },
```

SRC-06 openingSubscriptions.ts

File: SanctDemo/src/lib/openingSubscriptions.ts • extract • 118 lines in full

What this code does: The 'notify me' bells, stored per device.

Lines 45–49 of openingSubscriptions.ts

```
export function watchKey(watch: Watch): string {
  return watch.itemId
    ? `${watch.kind}:${watch.parishId}:${watch.itemId}`
    : `${watch.kind}:${watch.parishId}`;
}
```

Lines 79–86 of openingSubscriptions.ts

```
export function setWatched(watch: Watch, on: boolean): boolean {
  const keys = readAll();
  if (on) keys.add(watchKey(watch));
  else keys.delete(watchKey(watch));
  writeAll(keys);
  notifyListeners();
  return on;
}
```

3. Prayer, Narration and Speech

SRC-07 speech.ts

File: SanctDemo/src/lib/speech.ts • extract • 227 lines in full

What this code does: Selecting a voice engine, and why the phone cannot use the browser's.

Lines 64–65 of speech.ts

```
export function chooseEngine(
  { native, webUsable }: { native: boolean; webUsable: boolean },
```

Lines 113–140 of speech.ts

```
export function voiceFor(
  language: Language,
  win: Window | undefined = typeof window === 'undefined' ? undefined : window,
): SpeechSynthesisVoice | null {
  const synth = (win as (Window & { speechSynthesis?: SpeechSynthesis }) | undefined)?.speechSynthesis
  if (!synth || typeof synth.getVoices !== 'function') return null

  let voices: SpeechSynthesisVoice[]
  try {
    voices = synth.getVoices()
  } catch {
    return null
  }
  if (!voices || voices.length === 0) return null

  const wanted = language === 'fil' ? /^(fil|tl)\b/i : /^en\b/i
  const tagOf = (v: SpeechSynthesisVoice) => (v.lang ?? '').replace('_', '-')

  // An exact regional match first, then the bare language.
  const exact = language === 'fil' ? 'fil-ph' : 'en-us'
  return (
    voices.find(v => tagOf(v).toLowerCase() === exact) ??
    voices.find(v => wanted.test(tagOf(v))) ??
    null
  )
}
```

Lines 160–213 of speech.ts

```
export async function speakAloud(text: string, language: Language): Promise<void> {
  const engine = currentEngine()
  if (engine === 'none' || !text.trim()) return

  if (engine === 'native') {
    // Stop first. The plugin queues otherwise, and the pilgrim waits out
    // the previous station before hearing this one.
    await stopAloud()
    await TextToSpeech.speak({
      text,
```

```
      lang: speechLocale(language),
      rate: RATE,
      // Read through the media channel so the phone's volume keys do
      // what the pilgrim expects while the tour is talking.
      category: 'playback',
    })
    return
  }

  window.speechSynthesis.cancel()
  const utterance = new SpeechSynthesisUtterance(text)
  utterance.lang = speechLocale(language)
  // The voice object, not only the tag. Without this the engine reads
  // Tagalog in whatever its default voice is, which is English.
  const picked = voiceFor(language)
  if (picked) utterance.voice = picked
  utterance.rate = RATE

  // Resolve when the voice actually stops, not when it starts.
  await new Promise<void>(resolve => {
    let settled = false
    const done = () => {
      if (settled) return
      settled = true
      resolve()
    }
    utterance.onend = done
    // An engine that fails mid-sentence must not leave the caller
    // waiting forever on a promise that will never settle.
    utterance.onerror = done
    window.speechSynthesis.speak(utterance)
  })
}
```

SRC-08 DailyRosary.tsx

File: SanctDemo/src/components/DailyRosary.tsx • extract • 203 lines in full

What this code does: The bridge that lends the Rosary the phone's own voice and recogniser.

Lines 42–138 of DailyRosary.tsx

```
export default function DailyRosary() {
  const frame = useRef<HTMLIFrameElement | null>(null);

  /** Announce us again once the page is up, in case its hello raced us. */
  function announce() {
    frame.current?.contentWindow?.postMessage(
      { channel: CHANNEL, type: "host-ready" },
      window.location.origin,
    );
  }

  useEffect(() => {
    function reply(message: Record<string, unknown>) {
      const win = frame.current?.contentWindow;
      if (!win) return;
      win.postMessage({ channel: CHANNEL, ...message }, window.location.origin);
    }

    async function onMessage(event: MessageEvent) {
      // Same page, same frame, or it is not the Rosary talking.
      if (event.origin !== window.location.origin) return;
      if (frame.current && event.source !== frame.current.contentWindow) return;

      const data = event.data as
        | { channel?: string; type?: string; text?: string; lang?: string }
        | null;
      if (!data || data.channel !== CHANNEL) return;

      if (data.type === "hello") {
        reply({ type: "host-ready" });
        return;
      }

      if (data.type === "stop") {
        await stopAloud();
        return;
      }

      if (data.type === "speak" && typeof data.text === "string") {
        await speakAloud(data.text, data.lang === "fil" ? "fil" : "en");
        // Resolves when the voice stops, so this is the moment the page
        // can light its next bead rather than guess at a duration.
        reply({ type: "spoke-end" });
        return;
      }

      // ---- listening, for the same reason as speaking ----------------- The Rosary follows along as you pray…

      if (data.type === "listen-available") {
        reply({ type: "listen-availability", available: await listenAvailable() });
        return;
      }
```

```
      if (data.type === "listen-start") {
        await startListening(data.lang === "fil" ? "fil" : "en", reply);
        return;
      }

      if (data.type === "listen-stop") {
        await stopListening();
        return;
      }
    }

    window.addEventListener("message", onMessage);
    return () => {
      window.removeEventListener("message", onMessage);
      // Leaving the Pray tab must not leave a voice praying on behind it.
      void stopAloud();
    };
  }, []);

  return (
    <div className="flex-1 flex flex-col h-full min-h-[600px] w-full">
      <iframe
        ref={frame}
        src="/rosary/index.html"
        title="Holy Rosary"
        // Without this the WebView refuses getUserMedia inside the frame, which also kills the loudness glow that…
        allow="microphone"
        onLoad={announce}
        style={{ border: 0 }}
        className="flex-1 w-full h-full"
      />
    </div>
  );
}
```

Lines 104–108 of DailyRosary.tsx

```
        await startListening(data.lang === "fil" ? "fil" : "en", reply);
        return;
      }

      if (data.type === "listen-stop") {
```

4. Artificial Intelligence — Scanner and Parish Assistant

SRC-09 sanctiGrounding.ts

File: SanctDemo/src/lib/sanctiGrounding.ts • extract • 244 lines in full

What this code does: Grounding: what the assistant is allowed to claim, and what it must refuse.

Lines 83–113 of sanctiGrounding.ts

```
export function massFact(
  parishContent: ParishContent | null | undefined,
  compiled: MassScheduleEntry[] | null | undefined,
): Fact | Gap {
  const parish = parishContent?.massSchedule
  const usingParish = Boolean(parish && parish.length > 0)
  const chosen = usingParish ? parish : compiled
  const source: 'parish' | 'app' = usingParish ? 'parish' : 'app'

  if (!chosen || chosen.length === 0) {
    return {
      topic: 'mass',
      reason: 'No Mass schedule has been published for this parish.',
    }
  }

  const lines = chosen
    .map(entry => {
      const times = liveTimesFor(entry)
      if (times.length === 0) return `${entry.day}: no Mass currently scheduled`
      return `${entry.day}: ${times.join(', ')}`
    })
    .join('; ')

  return {
    topic: 'mass',
    value: lines,
    source,
    updatedAt: source === 'parish' ? parishContent?.updatedAt : undefined,
  }
}
```

Lines 182–195 of sanctiGrounding.ts

```
export function offersItem(
  parishContent: ParishContent | null | undefined,
  itemId: string,
): 'yes' | 'no' | 'unstated' {
  if (parishContent?.closedApplications?.includes(itemId)) return 'no'
  const content = parishContent?.itemContent?.[itemId]
  const published = Boolean(
    content?.about?.trim() ||
      content?.schedule?.trim() ||
      content?.requirements?.some(r => r.trim()) ||
      content?.process?.some(p => p.trim()),
  )
  return published ? 'yes' : 'unstated'
```

```
}
```

Lines 221–244 of sanctiGrounding.ts

```
export function groundingBlock(g: Grounding, now: Date = new Date()): string {
  const lines: string[] = [`PARISH: ${g.parishName}`]

  if (g.facts.length === 0) {
    lines.push('VERIFIED FACTS: none available for this parish.')
  } else {
    lines.push('VERIFIED FACTS (you may state these):')
    for (const f of g.facts) {
      const fresh = freshnessNote(f, now)
      const origin =
        f.source === 'parish'
          ? `parish office${fresh ? `, ${fresh}` : ''}`
          : 'SanctiWalk app data, not maintained by this parish'
      lines.push(`- [${f.topic}] ${f.value} (source: ${origin})`)
    }
  }

  if (g.gaps.length > 0) {
    lines.push('NOT AVAILABLE (you must say these are not available, never guess):')
    for (const gap of g.gaps) lines.push(`- [${gap.topic}] ${gap.reason}`)
  }

  return lines.join('\n')
}
```

SRC-10 askSancti.ts

File: SanctDemo/src/lib/askSancti.ts • extract • 84 lines in full

What this code does: The client call, which falls back to the on-device matcher on every failure.

Lines 50–84 of askSancti.ts

```
export async function askSancti(
  req: AskRequest,
  fetchImpl: typeof fetch = fetch,
): Promise<AskReply | null> {
  const controller = new AbortController()
  const timer = setTimeout(() => controller.abort(), TIMEOUT_MS)

  try {
    const res = await fetchImpl(apiUrl('/api/ask'), {
      method: 'POST',
      headers: withAppKey({ 'Content-Type': 'application/json' }),
      body: JSON.stringify(req),
      signal: controller.signal,
    })
    if (!res.ok) return null

    const data = await res.json()
    const answer = typeof data?.answer === 'string' ? data.answer.trim() : ''
    // An empty answer is a failure wearing a 200. Falling back to the
    // on-device reply is better than showing a blank message bubble.
    if (!answer) return null

    return {
      answer,
      grounded: Boolean(data?.grounded),
      sources: Array.isArray(data?.sources) ? data.sources : [],
      gaps: Array.isArray(data?.gaps) ? data.gaps : [],
    }
  } catch {
    // Offline, aborted, non-JSON body - all the same outcome.
    return null
  } finally {
    clearTimeout(timer)
  }
}
```

SRC-11 scanQuota.ts

File: SanctDemo/src/lib/scanQuota.ts • extract • 104 lines in full

What this code does: Limiting how many images one person may send for recognition.

Lines 57–60 of scanQuota.ts

```
export function isScannableParish(parishId: unknown): parishId is ScannableParish {
  return typeof parishId === 'string'
    && (SCANNABLE_PARISHES as readonly string[]).includes(parishId)
}
```

Lines 78–87 of scanQuota.ts

```
export function budgetFrom(used: number, parishId: string, day: string): ScanBudget {
  const safe = Number.isFinite(used) && used > 0 ? Math.floor(used) : 0
  return {
    limit: SCANS_PER_PARISH_PER_DAY,
    used: safe,
    remaining: Math.max(0, SCANS_PER_PARISH_PER_DAY - safe),
    day,
    parishId,
  }
}
```

5. Maps and Navigation

SRC-12 followCamera.ts

File: SanctDemo/src/lib/followCamera.ts • extract • 190 lines in full

What this code does: The Waze-style follow camera and its adaptive zoom.

Lines 169–176 of followCamera.ts

```
export function zoomForTurn(metersToManeuver: number | null): number {
  if (metersToManeuver === null || !Number.isFinite(metersToManeuver)) return FOLLOW_ZOOM
  if (metersToManeuver <= TURN_NEAR_M) return APPROACH_ZOOM
  if (metersToManeuver >= TURN_FAR_M) return CRUISE_ZOOM

  const t = (metersToManeuver - TURN_NEAR_M) / (TURN_FAR_M - TURN_NEAR_M)
  return APPROACH_ZOOM + (CRUISE_ZOOM - APPROACH_ZOOM) * t
}
```

Lines 188–190 of followCamera.ts

```
export function needsZoomChange(currentZoom: number, targetZoom: number): boolean {
  return Math.abs(currentZoom - targetZoom) >= ZOOM_EPSILON
}
```

Lines 82–95 of followCamera.ts

```
export function followCamera(
  position: Coordinates,
  heading: number | null,
  currentBearing: number,
  zoom: number = FOLLOW_ZOOM,
): FollowCamera {
  return {
    center: [position.lng, position.lat],
    bearing: heading ?? currentBearing,
    zoom,
    pitch: FOLLOW_PITCH,
    offset: [0, FOLLOW_OFFSET_Y],
  }
}
```

SRC-13 geo.ts

File: SanctDemo/src/lib/geo.ts • extract • 42 lines in full

What this code does: Distance and bearing between two coordinates.

Lines 14–25 of geo.ts

```
export function haversineMeters(a: Coordinates, b: Coordinates): number {
  const dLat = toRad(b.lat - a.lat)
  const dLng = toRad(b.lng - a.lng)
  const lat1 = toRad(a.lat)
  const lat2 = toRad(b.lat)

  const h =
    Math.sin(dLat / 2) ** 2 +
    Math.cos(lat1) * Math.cos(lat2) * Math.sin(dLng / 2) ** 2

  return 2 * EARTH_RADIUS_M * Math.asin(Math.sqrt(h))
}
```

Lines 27–42 of geo.ts

```
export function nearestLiveParish(
  pos: Coordinates | null,
  parishes: GeoParish[],
): { parish: GeoParish; distance: number } | null {
  if (!pos) return null

  let best: { parish: GeoParish; distance: number } | null = null
  for (const parish of parishes) {
    if (parish.status !== 'live') continue
    const distance = haversineMeters(pos, parish.coordinates)
    if (!best || distance < best.distance) {
      best = { parish, distance }
    }
  }
  return best
}
```

6. Backend and API

SRC-14 server.ts

File: SanctDemo/server.ts • extract • 1080 lines in full

What this code does: The guards every endpoint passes through: the shared key, the signed-in user, and the parish they are allowed to ask about.

Lines 200–209 of server.ts

```
function requireAppKey(req: express.Request, res: express.Response): boolean {
  if (!APP_KEY) return true;

  const provided = req.get(APP_KEY_HEADER);
  if (provided === APP_KEY) return true;

  console.warn(`[Auth] rejected ${req.method} ${req.path} from ${clientIp(req)} - bad or missing key`);
  res.status(401).json({ error: "This build is not authorised to use the SanctiWalk service." });
  return false;
}
```

Lines 280–314 of server.ts

```
async function requireUser(
  req: express.Request,
  res: express.Response,
): Promise<string | null> {
  if (!quotaStoreReady()) {
    // Fail CLOSED.
    if (!UNCOUNTED_OK) {
      res.status(503).json({
        error: "The scanner is not available right now. Please try again later.",
        code: "quota_unavailable",
      });
      return null;
    }
    if (!devBackstopAllows(req, res)) return null;
    return `dev:${clientIp(req)}`;
  }

  const uid = await uidFromToken(req.get("authorization"));
  if (uid) return uid;

  res.status(401).json({
    error: "Sign in to use the AI scanner. Your daily scans are counted against your account.",
    code: "sign_in_required",
  });
  return null;
}
```

Lines 324–333 of server.ts

```
function requireParish(req: express.Request, res: express.Response): string | null {
  const parishId = (req.body?.parishId ?? req.query?.parishId) as unknown;
```

```
  if (isScannableParish(parishId)) return parishId;

  res.status(400).json({
    error: "The scanner needs to know which parish you are visiting.",
    code: "parish_required",
  });
  return null;
}
```

SRC-15 server.ts

File: SanctDemo/server.ts • extract • 1080 lines in full

What this code does: The image recognition endpoint.

Lines 499–782 of server.ts

```
app.post("/api/identify", async (req, res) => {
  if (!requireAppKey(req, res)) return;

  const uid = await requireUser(req, res);
  if (!uid) return;
  const parishId = requireParish(req, res);
  if (!parishId) return;

  // Reserved here, before the model is asked.
  let spent = false;
  let budget: ScanBudget;

  try {
    const attempt = quotaStoreReady()
      ? await spendScan(uid, parishId)
      // The development path, and only reachable through it, because requireUser has already answered 503 for…
      : { allowed: true, budget: emptyBudget(parishId) };

    budget = attempt.budget;
    if (!attempt.allowed) {
      return res.status(429).json({
        error: spentMessage(parishName(parishId)),
        code: "daily_limit",
        budget,
      });
    }
    spent = quotaStoreReady();
  } catch (err) {
    console.error("[Quota] could not reserve a scan:", (err as Error).message);
    return res.status(503).json({
      error: "The scanner could not check your remaining scans. Please try again.",
      code: "quota_unavailable",
    });
  }

  try {
    const { imageBase64, mimeType, candidates } = req.body ?? {};

    if (typeof imageBase64 !== "string" || imageBase64.length < 512) {
      console.error(
        `[Scanner] Scanner error: rejected payload — imageBase64 was ${
          typeof imageBase64 === "string" ? `${imageBase64.length} chars` : typeof imageBase64
        }`,
      );
      return res.status(400).json({
        error: "The captured image did not arrive properly. Please try scanning again.",
      });
    }

    // Base64 is 4 chars per 3 bytes, so this is the decoded size.
    const imageBytes = Math.round(imageBase64.length * 0.75);
    console.log(`[Scanner] API request: ${(imageBytes / 1024).toFixed(0)} KB image, model ${VISION_MODEL}`);
```

```
const shortlist: string[] = Array.isArray(candidates) ? candidates.filter(Boolean) : [];

const instruction = shortlist.length
  ? [
      "You are identifying a feature inside a specific Catholic parish church.",
      "",
      "The pilgrim is at a station that is one of the following:",
      ...shortlist.map((name: string) => `- ${name}`),
      "",
      "Look carefully at the whole frame before answering.",
      "",
      "If the photo shows one of them, name that station and set",
      "matchedStation to true. The list is the strongest evidence you",
      "have about what this object is, so prefer a listed station over a",
      "guess when the image is consistent with it.",
      "",
      "If it shows none of them, DO NOT set recognized to false. Identify the",
      "subject anyway — as specifically as you can — and set matchedStation to",
      "false. A pilgrim pointing a camera at something outside the tour still",
      "deserves an answer.",
      "",
      "Set recognized to false only when the frame is too blurry or dark, or",
      "shows nothing identifiable at all.",
      "",
      "In `summary`, say what you actually saw that led to the answer, so",
      "the pilgrim can check it against what is in front of them. Set",
      "`confidence` honestly, and when the frame is unusable put a short",
      "practical instruction in `advice`.",
      "",
      "Only state facts you can see in the image or that are common knowledge",
      "about the subject. Do not invent parish-specific history, dates, donors,",
      "or names — if you do not know, say what is visible instead.",
      "",
      "If the subject depicts a saint, a blessed, or Our Lady under a title,",
      "also fill saintName, aboutSaint, and - only when you are certain -",
      "feastDay and patronage. `summary` explains what you saw; `aboutSaint`",
      "says who they were. Someone standing in front of a statue can already",
      "see the statue, so a description of it teaches them nothing. Leave",
      "feastDay and patronage empty rather than guessing: a wrong feast day",
      "sends somebody to church on the wrong morning.",
    ].join("\n")
  : OPEN_INSTRUCTION;

const ai = getAi();
const askModel = (prompt: string, model: string = VISION_MODEL, timeoutMs: number = ATTEMPT_TIMEOUT_MS) => ai.models.generateContent({
  model,
  contents: [
    {
      role: "user",
      parts: [
        { inlineData: { mimeType: mimeType || "image/jpeg", data: imageBase64 } },
        { text: prompt },
      ],
    },
  ],
  config: {
    // Bounds a single attempt.
    abortSignal: AbortSignal.timeout(timeoutMs),
    // Recognition is perception, not deliberation.
    thinkingConfig: { thinkingLevel: ThinkingLevel.MINIMAL },
    responseMimeType: "application/json",
    responseSchema: {
      type: Type.OBJECT,
      properties: {
```

```
            recognized: {
              type: Type.BOOLEAN,
              description: "False when the frame shows nothing identifiable.",
            },
            title: {
              type: Type.STRING,
              // This field's own description used to read "the subject's specific name", which pulled against the instruction…
              description:
                "What the subject is. Use a specific saint or devotion ONLY when the image shows attributes that distinguish it (an inscription, name plate, or unique emblem).
Otherwise use an accurate general description such as 'Crowned Marian image with the Child Jesus'. Empty when not recognized.",
            },
            category: { type: Type.STRING, description: "Short type label: Statue, Altar, Window, Painting, Relic, Architecture." },
            confidence: {
              type: Type.NUMBER,
              description:
                "0 to 1. Be honest. Use 0.9+ only when an inscription or unique emblem confirms the identification; use 0.4-0.6 for a general category recognised without
distinguishing marks.",
            },
            summary: {
              type: Type.STRING,
              description:
                "2-4 sentences for someone standing in front of it. Begin by naming the visual evidence you used — the crown, vestment colour, emblem, inscription — then what it is.
Say plainly if you are unsure.",
            },
            highlights: {
              type: Type.ARRAY,
              items: { type: Type.STRING },
              description: "Three short standalone facts.",
            },
            advice: {
              type: Type.STRING,
              description:
                "When recognized is false: one short, practical instruction for the pilgrim, e.g. move closer or improve the lighting. Empty otherwise.",
            },
            matchedStation: {
              type: Type.BOOLEAN,
              description:
                "True when the subject is one of the listed stations. False when it was identified but is not on the list. Ignored when no list was given.",
            },
            saintName: {
              type: Type.STRING,
              description:
                "When the subject depicts a saint, blessed, or Our Lady under a title, their full common English name, e.g. 'Saint Francis of Assisi' or 'Our Lady of Perpetual
Help'. Empty string when the subject is not a holy figure.",
            },
            feastDay: {
              type: Type.STRING,
              description:
                "That saint's feast day in the Roman calendar, e.g. 'October 4'. Leave EMPTY if you are not certain - a wrong feast day sends someone to church on the wrong morning.
Never guess.",
            },
            patronage: {
              type: Type.STRING,
              description:
                "What they are the patron of, in a few words, e.g. 'animals, ecology and merchants'. Leave EMPTY if unsure. Never guess.",
            },
            aboutSaint: {
              type: Type.STRING,
              description:
                "3-5 sentences on who this saint actually was and why they are venerated: when and where they lived, what they are known for, and what their usual emblems mean. This
is general Catholic knowledge about the person, NOT a description of the object in the photo - the pilgrim can already see the object. Write it in your own words. Empty string when
the subject is not a holy figure.",
            },
          },
```

```
        required: ["recognized", "title", "category", "confidence", "summary", "highlights"],
      },
    },
  });

  const startedAt = Date.now();
  // Retry sequencing lives in src/lib/modelRetry.ts, where it is unit tested — the behaviour that matters (three…
  const { value: response, model: answeringModel, attempt } = await askWithRetry(
    retryPlan(VISION_MODEL, VISION_MODEL_FALLBACK),
    (model, timeoutMs) => askModel(instruction, model, timeoutMs),
    {
      onBusy: (model, n) => console.warn(`[Scanner] ${model} did not answer (attempt ${n}) — retrying`),
    },
  );
  if (attempt > 1) console.log(`[Scanner] Recovered on attempt ${attempt} using ${answeringModel}`);

  // Logged so "the scanner feels slow" can be answered with a number
  // instead of a guess — which is how the 20s-to-1.5s fix was found.
  console.log(`[Scanner] API response: ${answeringModel} answered in ${Date.now() - startedAt}ms`);
  const text = response.text;
  if (!text) {
    return res.status(502).json({ error: "The vision model returned no content." });
  }

  const result = JSON.parse(text);

  // The station shortlist used to instruct the model to answer "none of these" for anything that was not one of…
  if (!shortlist.length) delete result.matchedStation;

  return res.json({ ...result, budget });
} catch (error: any) {
  const message = String(error?.message ?? error);

  // The free tier allows a small number of requests per model per day. Say so
  // plainly rather than surfacing a raw stack trace to a pilgrim.
  if (message.includes("429") || message.toUpperCase().includes("RESOURCE_EXHAUSTED")) {
    if (spent) await refundScan(uid, parishId);
    return res.status(429).json({
      error: "The recognition service is busy. Please try again in a little while.",
      code: "quota_exhausted",
      budget: await readBudget(uid, parishId),
    });
  }

  // The real error goes to the console; the pilgrim gets something they can act on.
  console.error("[Scanner] Scanner error:", message);

  // Everything below is the server, the key or the model failing - none of it is the visitor's doing, so they get…
  if (spent) await refundScan(uid, parishId);

  if (/API key|API_KEY|PERMISSION_DENIED|UNAUTHENTICATED/i.test(message)) {
    return res.status(500).json({
      error: "Recognition is not configured on the server. Check GEMINI_API_KEY.",
      code: "auth",
    });
  }
  if (/deadline|timeout|ETIMEDOUT|ECONNRESET|fetch failed|ENOTFOUND/i.test(message)) {
    return res.status(504).json({
      error: "Recognition timed out. Check your connection and try again.",
      code: "timeout",
    });
  }
  // Congestion, after every retry and the fallback model have been tried.
  if (isModelBusy(message) || isTimeout(message)) {
```

```
      return res.status(503).json({
        error:
          "The recognition service is busy right now. This is on their side, not yours — wait a few seconds and tap scan again.",
        code: "model_busy",
      });
    }

    return res.status(500).json({ error: "Recognition failed. Please try again." });
  }
});
```

SRC-16 server.ts

File: SanctDemo/server.ts • extract • 1080 lines in full

What this code does: The grounded parish assistant endpoint.

Lines 841–943 of server.ts

```
app.post("/api/ask", async (req, res) => {
  if (!requireAppKey(req, res)) return;

  const question = typeof req.body?.question === "string" ? req.body.question.trim() : "";
  if (!question) return res.status(400).json({ error: "Missing question." });
  // A guard, not a limit on curiosity: the grounding block and the
  // recent conversation share a context window with it.
  if (question.length > 600) return res.status(400).json({ error: "Question is too long." });

  const parishId = typeof req.body?.parishId === "string" ? req.body.parishId : "";
  const parishName =
    typeof req.body?.parishName === "string" && req.body.parishName.trim()
      ? req.body.parishName.trim()
      : "this parish";

  try {
    const ai = getAi();

    // Retrieval first. Everything the answer may assert is gathered
    // here, from the parish's own document where it has one.
    const content = parishId ? await readParishContent(parishId) : null;
    const facts: Fact[] = [];
    const gaps: Gap[] = [];

    if (parishId) {
      const mass = massFact(content, MASS_SCHEDULES[parishId]?.schedule);
      if (isGap(mass)) gaps.push(mass); else facts.push(mass);

      if (content?.description?.trim()) {
        facts.push({ topic: "description", value: content.description.trim(), source: "parish", updatedAt: content.updatedAt });
      }
      if (content?.historyBody?.trim()) {
        facts.push({ topic: "history", value: content.historyBody.trim(), source: "parish", updatedAt: content.updatedAt });
      } else {
        gaps.push({ topic: "history", reason: "This parish has not published its history in the app." });
      }

      // Only the sacraments and ministries the question actually reaches for.
      const asked = question.toLowerCase();
      const TOPICS: Array<[string, string, string[]]> = [
        ["sac-baptism", "Baptism", ["bapti", "binyag", "christen"]],
        ["sac-wedding", "Wedding", ["wedding", "marriage", "kasal", "ikakasal"]],
        ["sac-confirmation", "Confirmation", ["confirmation", "kumpil"]],
        ["sac-eucharist", "Holy Communion", ["communion", "eucharist", "komunyon"]],
        ["min-choir", "Choir", ["choir", "koro", "sing"]],
        ["min-altar-servers", "Altar Servers", ["altar server", "sakristan", "acolyte"]],
      ];
      for (const [id, label, words] of TOPICS) {
        if (!words.some(w => asked.includes(w))) continue;
        const { facts: f, gaps: g } = itemFacts(content, id, label);
        facts.push(...f);
        gaps.push(...g);
      }
```

```
    }

    const context = parishId
      ? groundingBlock({ parishId, parishName, facts, gaps })
      : [
          "NO PARISH SELECTED.",
          "You have no verified parish information for this question.",
          "If the question needs parish-specific facts, ask which parish they mean instead of guessing.",
        ].join("\n");

    // Recent turns only, and only as plain text, so the model can
    // resolve "what about tomorrow" without the whole session.
    const history = Array.isArray(req.body?.history) ? req.body.history.slice(-6) : [];
    const transcript = history
      .filter((m: unknown) => typeof (m as { text?: unknown })?.text === "string")
      .map((m: { role?: string; text: string }) => (m.role === "sancti" ? "Sancti: " : "User: ") + m.text)
      .join("\n");

    const prompt = [
      context,
      transcript ? "\nRECENT CONVERSATION:\n" + transcript : "",
      "\nUSER QUESTION: " + question,
    ].join("\n");

    const response = await ai.models.generateContent({
      model: "gemini-3.5-flash",
      contents: prompt,
      config: { systemInstruction: ASK_SYSTEM_INSTRUCTION, temperature: 0.2 },
    });

    const answer = (response.text ?? "").trim();
    if (!answer) return res.status(502).json({ error: "The assistant did not answer." });

    res.json({
      answer,
      grounded: facts.length > 0,
      // So the UI can say where this came from without re-deriving it.
      sources: facts.map(f => ({ topic: f.topic, source: f.source, updatedAt: f.updatedAt ?? null })),
      gaps: gaps.map(g => g.topic),
    });
  } catch (err) {
    // Never a fabricated fallback answer: the client has a real on-device assistant to fall back to, and it is…
    console.warn("[Sancti] ask failed:", (err as Error).message);
    res.status(503).json({ error: "The assistant is unavailable." });
  }
});
```

7. Firebase, Database and Security

SRC-17 firebase.ts

File: SanctDemo/src/lib/firebase.ts • whole file • 11 lines in full

What this code does: Firebase initialisation.

```
import { initializeApp, getApps, getApp } from "firebase/app";
import { getAuth } from "firebase/auth";
import { getFirestore } from "firebase/firestore";
import firebaseConfig from "../../firebase-applet-config.json";

// Initialize Firebase
const app = getApps().length === 0 ? initializeApp(firebaseConfig) : getApp();
export const auth = getAuth(app);
export const db = getFirestore(app, firebaseConfig.firestoreDatabaseId);

export default app;
```

SRC-18 authFlow.ts

File: SanctDemo/src/lib/authFlow.ts • extract • 262 lines in full

What this code does: Authentication.

Lines 172–185 of authFlow.ts

```
export async function signIn(email: string, password: string): Promise<SignInResult> {
  try {
    const cred = await signInWithEmailAndPassword(auth, email, password);
    const profile = await getProfile(cred.user.uid);

    // Administrators always take the second factor. They can change other
    // people's records, so their password alone is worth more.
    const required = profile?.role === "church_admin" || cred.user.emailVerified;

    return { user: cred.user, pendingSecondFactor: required };
  } catch (err) {
    throw friendlyAuthError(err);
  }
}
```

Lines 259–262 of authFlow.ts

```
export async function signOutNow(): Promise<void> {
  window.localStorage.removeItem(PENDING_EMAIL_KEY);
  await signOut(auth);
}
```

SRC-19 availability.ts

File: SanctDemo/src/lib/availability.ts • extract • 102 lines in full

What this code does: Whether a ministry or sacrament accepts applications.

Lines 34–36 of availability.ts

```
export function closedIds(content: ParishContent | null | undefined): string[] {
  return content?.closedApplications ?? [];
}
```

Lines 45–50 of availability.ts

```
export function isOpenForApplications(
  content: ParishContent | null | undefined,
  itemId: string,
): boolean {
  return !closedIds(content).includes(itemId);
}
```

Lines 60–69 of availability.ts

```
export function withAvailability(
  current: string[],
  itemId: string,
  open: boolean,
): string[] {
  const next = new Set(current);
  if (open) next.delete(itemId);
  else next.add(itemId);
  return [...next].sort();
}
```

SRC-20 firestore.rules

File: SanctDemo/firestore.rules • whole file • 409 lines in full

What this code does: The database security rules — who may read and write what. Included in full because every line is a access decision.

```
rules_version = '2';

// Firestore security rules for SanctiWalk.

service cloud.firestore {
  match /databases/{database}/documents {

    // A churchId that cannot collide with a real one, for documents written before churches existed.
    function noChurch() {
      return '__none__';
    }

    function isSignedIn() {
      return request.auth != null;
    }

    function me() {
      return get(/databases/$(database)/documents/users/$(request.auth.uid)).data;
    }

    function myRole() {
      return isSignedIn() ? me().get('role', 'user') : 'none';
    }

    function myChurch() {
      return isSignedIn() ? me().get('churchId', noChurch()) : noChurch();
    }

    // A church admin acting on their OWN church.
    function isAdminOf(churchId) {
      return isSignedIn()
        && churchId != noChurch()
        && churchId == myChurch()
        && myRole() == 'church_admin';
    }

    // The legacy global administrator, from before churches existed.
    function isSuperAdmin() {
      return isSignedIn()
        && exists(/databases/$(database)/documents/admins/$(request.auth.uid));
    }

    // ---------------------------------------------------------------- users

    // The fields a person may change about themselves.
    function onlyEditableUserFields() {
      return request.resource.data.diff(resource.data).affectedKeys()
        .hasOnly([
          'fullName', 'nickname', 'phoneNumber', 'photoUrl', 'updatedAt',
          'preferredLanguage', 'notificationsSeenAt',
          // The activity heartbeat.
          'lastActiveAt',
          // The pilgrimage progress the app saves as you walk.
          'points', 'steps', 'distanceKm',
          'completedStations', 'completedRoutes', 'badges'
        ]);
```

```
}

match /users/{userId} {
  // A church admin may read their own parishioners, for the admin user
  // list. Nobody reads across churches.
  allow read: if isSignedIn()
    && (request.auth.uid == userId
        || isAdminOf(resource.data.get('churchId', noChurch()))
        || isSuperAdmin());

  // At creation a user may declare which church they joined, but not
  // that they run it, and not that they are already verified.
  allow create: if isSignedIn()
    && request.auth.uid == userId
    && request.resource.data.get('role', 'user') == 'user'
    && request.resource.data.get('phoneVerified', false) == false;

  allow update: if isSignedIn()
    && request.auth.uid == userId
    && onlyEditableUserFields();

  // Profiles are never deleted from the client; an account removal is an
  // auth operation with records hanging off it.
  allow delete: if false;
}

// ------------------------------------------------------------- churches

// Public to read: the signup screen lists churches before anyone has signed in.
match /churches/{churchId} {
  allow read: if true;

  allow update: if isAdminOf(churchId)
    && request.resource.data.adminUid == resource.data.adminUid
    && request.resource.data.id == resource.data.id;

  allow create, delete: if false;
}

// ------------------------------------------------------- applications

// Sacrament and ministry applications share one collection, with `kind` separating them, because every query…

// What an applicant may assert when submitting.
function closedApplications(churchId) {
  return exists(/databases/$(database)/documents/parishContent/$(churchId))
    ? get(/databases/$(database)/documents/parishContent/$(churchId))
        .data.get('closedApplications', [])
    : [];
}

function validNewApplication() {
  return isSignedIn()
    && request.resource.data.uid == request.auth.uid
    && request.resource.data.status == 'pending'
    && request.resource.data.kind in ['sacrament', 'ministry']
    // The church you belong to is the church you apply to.
    && request.resource.data.churchId == myChurch()
    && myChurch() != noChurch()
    // Which ministry or sacrament this is, by id.
    && request.resource.data.itemId is string
    && request.resource.data.itemId != ''
    // Closed means closed.
    && !(request.resource.data.itemId in closedApplications(myChurch()));
```

```
}

// What a reviewer may change: the decision and its audit trail. Not the
// applicant, not the church, not the submitted answers.
function onlyStatusFields() {
  return request.resource.data.diff(resource.data).affectedKeys()
    .hasOnly(['status', 'adminNote', 'reviewedBy', 'reviewedAt',
              'updatedAt', 'history']);
}

match /applications/{applicationId} {
  allow create: if validNewApplication();

  allow read: if (isSignedIn() && resource.data.uid == request.auth.uid)
    || isAdminOf(resource.data.get('churchId', noChurch()))
    || isSuperAdmin();

  // An applicant cannot edit a submitted application.
  allow update: if isAdminOf(resource.data.get('churchId', noChurch()))
    && onlyStatusFields()
    && request.resource.data.status in
         ['pending', 'under_review', 'approved', 'rejected', 'completed'];

  // Withdrawal, not deletion.
  allow delete: if isSignedIn()
    && resource.data.uid == request.auth.uid
    && resource.data.status == 'pending';
}

// ------------------------------------------------------- notifications

match /notifications/{notificationId} {
  allow read: if isSignedIn() && resource.data.userId == request.auth.uid;

  allow create: if isSignedIn()
    && (request.resource.data.userId == request.auth.uid
        || isAdminOf(request.resource.data.get('churchId', noChurch())));

  // The only thing a reader may do to a notification is mark it read.
  allow update: if isSignedIn()
    && resource.data.userId == request.auth.uid
    && request.resource.data.diff(resource.data).affectedKeys()
         .hasOnly(['readAt']);

  allow delete: if isSignedIn() && resource.data.userId == request.auth.uid;
}

// ------------------------------------------------------------ activity

// Sign-ins, station check-ins and comments.
match /activity/{entryId} {
  allow create: if isSignedIn()
    && request.resource.data.uid == request.auth.uid;
  allow read: if (isSignedIn() && resource.data.uid == request.auth.uid)
    || isAdminOf(resource.data.get('churchId', noChurch()))
    || isSuperAdmin();
  allow update, delete: if false;
}

// ------------------------------------------------------------- visits

// Estimated church visits, for the parish dashboard.
// Page opens and downloads.
match /pageEvents/{eventId} {
```

```
  allow create: if request.resource.data.keys().hasOnly(['kind', 'visitorId', 'createdAt'])
    && request.resource.data.kind in ['open', 'download']
    && request.resource.data.visitorId is string
    && request.resource.data.visitorId.size() >= 8
    && request.resource.data.visitorId.size() <= 64
    // A window rather than an exact match on request.time.
    && request.resource.data.createdAt is timestamp
    && request.resource.data.createdAt > request.time - duration.value(5, 'm')
    && request.resource.data.createdAt < request.time + duration.value(5, 'm');

  allow read: if isSignedIn() && myRole() == 'church_admin';
  allow update, delete: if false;
}

match /visits/{visitId} {
  allow create: if isSignedIn()
    && request.resource.data.uid == request.auth.uid
    && request.resource.data.churchId is string
    && request.resource.data.churchId != ''
    && exists(/databases/$(database)/documents/churches/$(request.resource.data.churchId))
    && request.resource.data.keys().hasOnly(['uid', 'churchId', 'at', 'createdAt'])
    // The server's clock, not the handset's. A device with a wrong
    // date cannot backdate or postdate its way into another period.
    && request.resource.data.createdAt == request.time;

  // A pilgrim may read their own; an admin may read their parish's.
  // Nobody reads across parishes.
  allow read: if isSignedIn()
    && (resource.data.uid == request.auth.uid
        || isAdminOf(resource.data.get('churchId', noChurch()))
        || isSuperAdmin());

  allow update, delete: if false;
}

// ------------------------------------------------ diocese-wide content

// Split per operation rather than one `write`.
match /announcements/{docId} {
  allow read: if true;
  allow create: if isSuperAdmin()
    || isAdminOf(request.resource.data.get('churchId', noChurch()));
  allow update: if isSuperAdmin()
    || (isAdminOf(resource.data.get('churchId', noChurch()))
        && isAdminOf(request.resource.data.get('churchId', noChurch())));
  allow delete: if isSuperAdmin()
    || isAdminOf(resource.data.get('churchId', noChurch()));
}

// Admin-managed parish content: patron photo, description, Mass times, brand colour.
match /parishContent/{parishId} {
  allow read: if true;
  allow write: if isSuperAdmin() || isAdminOf(parishId);
}

// The legacy admin roster. Readable so the client can tell whether to
// offer the portal, never writable from the client.
match /admins/{userId} {
  allow read: if isSignedIn();
  allow write: if false;
}

// The scanner's daily allowance, one document per user per parish per day.
match /scanQuotas/{quotaId} {
```

```
      allow read, write: if false;
    }

    // Anything not named above is closed. Without this, a collection added
    // later is world-writable until someone remembers to write a rule for it.
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

8. Administrative Portal

SRC-21 AdminItems.tsx

File: SanctDemo/src/admin/AdminItems.tsx • extract • 201 lines in full

What this code does: Ministries and sacraments: editing the description and switching applications on or off.

Lines 39–159 of AdminItems.tsx

```
export default function AdminItems({
  session,
  kind,
}: {
  session: AdminSession;
  kind: "ministry" | "sacrament";
}) {
  const churchId = session.profile.churchId;
  const managed = useParishContent(churchId);

  // This parish's ministries, not every parish's. Sacraments are the
  // same seven everywhere, which is why they are not filtered.
  const items =
    kind === "ministry"
      ? ministriesForParish(churchId).map(m => ({ id: m.id, name: m.name }))
      : SACRAMENTS.map(s => ({ id: s.id, name: s.name }));

  const [openId, setOpenId] = useState<string | null>(null);
  const [busy, setBusy] = useState<string | null>(null);
  const [switching, setSwitching] = useState<string | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [saved, setSaved] = useState<string | null>(null);

  async function saveContent(itemId: string, patch: ItemContent) {
    setBusy(itemId);
    setError(null);
    try {
      await saveParishContent(
        churchId,
        { itemContent: mergeItemContent(managed, itemId, patch) },
        session.user.email ?? "",
      );
      setSaved(itemId);
      window.setTimeout(() => setSaved(null), 2600);
    } catch {
      setError("That could not be saved. Check your connection and try again.");
    } finally {
      setBusy(null);
    }
  }

  async function setOpen(itemId: string, open: boolean) {
    setSwitching(itemId);
    setError(null);
    try {
      await saveParishContent(
        churchId,
        { closedApplications: withAvailability(closedIds(managed), itemId, open) },
        session.user.email ?? "",
      );
```

```
  } catch {
    setError("That could not be saved. Check your connection and try again.");
  } finally {
    setSwitching(null);
  }
}

const openCount = items.filter(i => isOpenForApplications(managed, i.id)).length;
const title = kind === "ministry" ? "Ministries" : "Sacraments";

return (
  <div className="min-h-screen bg-[var(--color-brand-bg)]">
    <div className="max-w-[760px] mx-auto px-4 py-6">
      <AdminNav />

      <h1 className="text-[24px] font-bold font-serif italic text-[var(--color-brand-text)]">
        {title}
      </h1>
      <p className="text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
        Open a row to change what people read about it. Use the switch to
        decide whether they can apply. Switched off, it stays visible in the
        app with everything you have written, and the Apply button goes.
      </p>

      <p className="mt-3 text-[14px] font-semibold text-[var(--color-brand-secondary)]">
        {openCount} of {items.length} accepting applications
      </p>

      {error && (
        <p role="alert" className="mt-3 text-[14px] font-semibold text-[var(--color-brand-danger)]">
          {error}
        </p>
      )}

      <ul className="mt-4 divide-y divide-[var(--color-brand-border)] rounded-2xl border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] overflow-hidden">
        {items.map(item => {
          const open = isOpenForApplications(managed, item.id);
          return (
            <Item
              key={`${item.id}-${openId === item.id}`}
              id={item.id}
              name={item.name}
              kind={kind}
              edited={hasParishContent(managed, item.id)}
              open={openId === item.id}
              busy={busy === item.id}
              saved={saved === item.id}
              current={
                kind === "ministry"
                  ? resolveMinistry(managed, item.id, "en")
                  : resolveSacrament(managed, item.id, "en")
              }
              existing={managed?.itemContent?.[item.id] ?? {}}
              onToggle={() => setOpenId(openId === item.id ? null : item.id)}
              onSave={patch => void saveContent(item.id, patch)}
              availability={
                <AvailabilitySwitch
                  name={item.name}
                  open={open}
                  busy={switching === item.id}
                  onChange={next => void setOpen(item.id, next)}
                />
              }
            />
```

```
            );
          })}
        </ul>
      </div>
    </div>
  );
}
```

SRC-22 pageEvents.ts

File: SanctDemo/src/lib/pageEvents.ts • extract • 185 lines in full

What this code does: Counting opens and downloads without identifying anybody.

Lines 54–66 of pageEvents.ts

```
export function visitorId(): string {
  try {
    const existing = localStorage.getItem(VISITOR_KEY)
    if (existing) return existing
    const made = randomId()
    localStorage.setItem(VISITOR_KEY, made)
    return made
  } catch {
    // Storage refused. A per-session id still separates this visitor
    // from the others in the same window.
    return randomId()
  }
}
```

Lines 97–106 of pageEvents.ts

```
export function shouldRecord(
  kind: PageEventKind,
  now: number = Date.now(),
  read: (key: string) => string | null = safeGet,
): boolean {
  if (kind === 'download') return true
  const last = Number(read(lastKey(kind)) ?? '')
  if (!Number.isFinite(last) || last <= 0) return true
  return now - last >= OPEN_DEDUPE_MS
}
```

Lines 142–159 of pageEvents.ts

```
export function summarisePageEvents(events: PageEvent[]): PageEventTotals {
  const openers = new Set<string>()
  const downloaders = new Set<string>()
  let opens = 0
  let downloads = 0

  for (const e of events) {
    if (e.kind === 'open') {
      opens += 1
      if (e.visitorId) openers.add(e.visitorId)
    } else if (e.kind === 'download') {
      downloads += 1
      if (e.visitorId) downloaders.add(e.visitorId)
    }
  }

  return { opens, downloads, people: openers.size, downloaders: downloaders.size }
}
```

9. Map Placer and Station Pipeline

SRC-23 deploy-stations.py

File: SanctiWalkAR/Tools/deploy-stations.py • extract • 302 lines in full

What this code does: Turning placed stations into the table the AR app compiles in, including the per-scan handedness fix.

Lines 112–116 of deploy-stations.py

```
def mirror(v, site):
    """A map-space triple, with this site's sideways axis correction."""
    return (MIRROR_X.get(site, MIRROR_X_DEFAULT) * v[0], v[1], v[2])
```

Lines 158–219 of deploy-stations.py

```
def emit(site, field, parish_id, title, plan):
    lines = []
    lines.append("    /// <summary>")
    lines.append("    /// %s, %d stations." % (title, len(plan["stations"])))
    lines.append("    ///")
    lines.append("    /// Generated by Tools/deploy-stations.py from")
    lines.append("    /// Tools/viewer/out/%s.json. Do not edit by hand - the next" % site)
    lines.append("    /// save from the placer overwrites it.")
    lines.append("    ///")
    lines.append("    /// Positions are MultiSet map coordinates for %s," % plan.get("scan", "?"))
    lines.append("    /// used verbatim on the VPS route.")
    lines.append("    /// </summary>")
    lines.append("    public static readonly MuseumStation[] %s =" % field)
    lines.append("    {")

    i18n = load_i18n(site)
    stations = plan["stations"]
    for i, st in enumerate(stations):
        nxt = stations[i + 1]["id"] if i + 1 < len(stations) else ""
        x, y, z = mirror(st["map"], site)
        kind = st.get("kind", "history")
        lines.append("        new MuseumStation")
        lines.append("        {")
        lines.append("            stationId = %s," % cs(st["id"]))
        lines.append("            stationName = %s," % cs(st["name"]))
        lines.append("            subtitle = %s," % cs("%d ng %d" % (i + 1, len(stations))))
        lines.append("            kind = MuseumStationKind.%s," % kind.capitalize())
        lines.append("            position = new Vector3(%.3ff, %.3ff, %.3ff)," % (x, y, z))
        lines.append("            triggerRadius = %.2ff," % st.get("radius", 2.0))
        if kind == "model":
            lines.append("            model = %s," % cs(st.get("model", "")))
            lines.append("            modelScale = %.2ff," % st.get("modelScale", 1.0))
            lines.append("            yawDegrees = %.0ff," % st.get("yawDegrees", 0))
        else:
            # Resources.Load takes a path RELATIVE TO Resources/, so the subfolder is part of the name.
            img = st.get("image", "")
            lines.append("            image = %s," % cs("ArImages/" + img if img else ""))
        if st.get("caption"):
            lines.append("            shortDescription = %s," % cs(st["caption"]))
        if st.get("description"):
```

```
        lines.append("            detailedDescription = %s," % cs(st["description"]))

    # The same text in both languages, where it has been translated.
    tr = i18n.get(st["id"], {})
    for key, tl_field, en_field in (("name", "nameTl", "nameEn"),
                                    ("caption", "shortTl", "shortEn"),
                                    ("description", "detailedTl", "detailedEn")):
        pair = tr.get(key) or {}
        if pair.get("tl"):
            lines.append("            %s = %s," % (tl_field, cs(pair["tl"])))
        if pair.get("en"):
            lines.append("            %s = %s," % (en_field, cs(pair["en"])))
    lines.append("            nextStationId = %s," % cs(nxt))
    lines.append("        },")
lines.append("    };")
return "\n".join(lines)
```

SRC-24 save_server.py

File: SanctiWalkAR/Tools/viewer/save_server.py • whole file • 77 lines in full

What this code does: The local server behind the placer.

```
"""Serves the viewer and accepts PUT so the page can write files to disk.

The reference mesh arrives from MultiSet as Draco-compressed glTF, which
Unity cannot import without adding packages to a project that currently
builds cleanly. Rather than risk that, the decoding happens in the browser
- which already has a Draco decoder for the picker - and the result is
PUT back here as a plain OBJ that Unity imports natively.

Writes are confined to Tools/viewer/out.
"""

import http.server
import os
import posixpath
import socketserver

PORT = 8777
HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(HERE, "out")

class Handler(http.server.SimpleHTTPRequestHandler):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, directory=HERE, **kwargs)

    def do_PUT(self):
        name = posixpath.basename(self.path)
        if not name or name.startswith("."):
            self.send_error(400, "bad name")
            return

        length = int(self.headers.get("Content-Length", 0))
        os.makedirs(OUT, exist_ok=True)
        target = os.path.join(OUT, name)

        remaining = length
        with open(target, "wb") as handle:
            while remaining > 0:
                chunk = self.rfile.read(min(1 << 20, remaining))
                if not chunk:
                    break
                handle.write(chunk)
                remaining -= len(chunk)

        print(f"wrote {target} ({os.path.getsize(target)} bytes)", flush=True)
        self.send_response(200)
        self.send_header("Content-Length", "2")
        self.end_headers()
        self.wfile.write(b"ok")

    def end_headers(self):
        # Never cache anything here.
        self.send_header("Cache-Control", "no-store, must-revalidate")
        self.send_header("Pragma", "no-cache")
        self.send_header("Expires", "0")
        super().end_headers()
```

```
    def log_message(self, fmt, *args):
        pass

class Server(socketserver.ThreadingTCPServer):
    allow_reuse_address = True
    daemon_threads = True

if __name__ == "__main__":
    os.makedirs(OUT, exist_ok=True)
    with Server(("127.0.0.1", PORT), Handler) as httpd:
        print(f"serving {HERE} on http://127.0.0.1:{PORT}  (PUT -> {OUT})", flush=True)
        httpd.serve_forever()
```

10. Unity Augmented Reality

SRC-25 MarkerScan.cs

File: SanctiWalkAR/Assets/Scripts/MarkerScan.cs • whole file • 83 lines in full

What this code does: What a successful trigger-image scan produces, and what it deliberately does not.

```
using System;
using UnityEngine;
using UnityEngine.XR.ARSubsystems;

// <summary> Everything a successful trigger-image scan produces, and nothing else.
[Serializable]
public struct MarkerScan
{
    // ---- from the scan ------------------------------------------------
    // <summary>Reference image name that matched, e.g. "entrance".</summary>
    public string trackedImageName;

    // <summary>The library entry's guid, as ARCore reports it.</summary>
    public string referenceGuid;

    // <summary> The marker's pose in SESSION space - ARCore's own frame, whose origin is wherever the app happened…
    public Pose pose;

    // <summary>Tracking, Limited or None at the moment of the report.</summary>
    public TrackingState trackingState;

    // ---- from the reference image library ----------------------------
    // <summary>Printed size in metres. Configuration, not measurement.</summary>
    public Vector2 physicalSize;

    // ---- provenance ---------------------------------------------------
    // <summary>True when this came from ScanSimulator rather than a camera.</summary>
    public bool simulated;

    // <summary>Where the camera was when the image was reported.</summary>
    public Pose cameraPoseAtDetection;

    public override string ToString()
    {
        var e = pose.rotation.eulerAngles;
        return
            $"MarkerScan {{\n" +
            $"  source            : {(simulated ? "SIMULATED" : "ARCore camera")}\n" +
            $"  trackedImageName  : \"{trackedImageName}\"        <- from scan\n" +
            $"  referenceGuid     : {referenceGuid}               <- from scan\n" +
            $"  trackingState     : {trackingState}               <- from scan\n" +
            $"  pose.position     : ({pose.position.x:F3}, {pose.position.y:F3}, {pose.position.z:F3})  <- from scan, SESSION space, metres\n" +
            $"  pose.rotation     : euler ({e.x:F1}, {e.y:F1}, {e.z:F1})   <- from scan\n" +
            $"  physicalSize      : {physicalSize.x:F3} x {physicalSize.y:F3} m   <- from ChurchMarkers.asset, NOT the scan\n" +
            $"  cameraAtDetection : ({cameraPoseAtDetection.position.x:F3}, {cameraPoseAtDetection.position.y:F3}, {cameraPoseAtDetection.position.z:F3})\n" +
            $"}}";
    }
}
```

SRC-26 ChurchOriginTest.cs

File: SanctiWalkAR/Assets/Scripts/ChurchOriginTest.cs • extract • 2602 lines in full

What this code does: Establishing where the church is, rejecting the other parish's marker, and switching parish on a deep link.

Lines 498–526 of ChurchOriginTest.cs

```
public MuseumStation NearestStation()
{
    var all = Stations;
    if (all == null || all.Length == 0 || m_Camera == null) return null;

    MuseumStation nearest = null;
    var best = float.MaxValue;
    foreach (var s in all)
    {
        var d = MetresToStation(s);
        if (d < best) { best = d; nearest = s; }
    }

    // Still on the one we were on? Keep it unless beaten clearly.
    if (m_StickyStation != null)
    {
        var held = float.MaxValue;
        foreach (var s in all)
            if (s.stationId == m_StickyStation.stationId) { held = MetresToStation(s); break; }

        // Out of range entirely is not a tie to break - it is gone.
        var stillHere = held <= m_StickyStation.triggerRadius + k_StickyMarginMetres;
        if (stillHere && best > held - k_StickyMarginMetres)
            return m_StickyStation;
    }

    m_StickyStation = nearest;
    return nearest;
}
```

Lines 1096–1110 of ChurchOriginTest.cs

```
bool MarkerBelongsHere(ARTrackedImage image)
{
    if (!ParishMarker(out _, out _, out var expected) || string.IsNullOrEmpty(expected))
        return true;

    var seen = image.referenceImage.name;
    if (seen == expected)
        return true;

    Debug.LogWarning(
        $"[ChurchOriginTest] ignoring '{seen}' marker: {ParishId} is located " +
        $"from the '{expected}' sheet. Wrong church's print.");
    m_Status = $"That is the {seen} marker. Point at the {expected} one.";
    return false;
}
```

Lines 1112–1127 of ChurchOriginTest.cs

```
bool TryOrigin(System.Collections.Generic.IEnumerable<ARTrackedImage> images)
{
    foreach (var image in images)
    {
        if (image.trackingState != TrackingState.Tracking)
            continue;

        if (!MarkerBelongsHere(image))
            continue;

        SetOrigin(FromTrackedImage(image));
        return true;
    }

    return false;
}
```

Lines 1654–1752 of ChurchOriginTest.cs

```
void SetOrigin(MarkerScan scan)
{
    // A parish marker carries the whole map with it.
    if (ParishMarker(out var markerInMap, out var markerNormal, out var expected)
        && scan.trackedImageName == expected)
    {
        var seenFrom = scan.cameraPoseAtDetection.position;
        if (seenFrom == Vector3.zero && m_Camera != null)
            seenFrom = m_Camera.transform.position;

        var sessionFromMap = MapFromMarker(
            markerInMap, markerNormal, scan.pose, seenFrom);

        Debug.Log($"[ChurchOriginTest] marker '{expected}' places {ParishId} " +
                  $"from map {markerInMap} -> session {sessionFromMap.position}");

        // The sign hangs on the sheet, so the museum needs the sheet's own map pose - not the map origin, which is…
        if (m_Museum != null) m_Museum.SetMarkerSign(markerInMap, markerNormal);

        m_Origin = sessionFromMap;
        m_OriginSet = true;
        m_FromVps = true;        // map-v1: the frame is the map, as with VPS
        m_FromFloor = false;
        m_ToSession = MapSpaceIdentity;

        if (m_Images != null) m_Images.enabled = false;
        if (m_Readiness != null) m_Readiness.StopScanning();

        m_Status = $"Located from the {expected} marker";
        if (m_Museum != null) m_Museum.SetVisible(true);
        BindWorld();
        return;
    }

    m_Origin = scan.pose;
    m_OriginSet = true;

    // The marker route skips the checklist, so the UI has to come back.
    if (m_Museum != null)
```

```
            m_Museum.SetVisible(true);

        // Which side of the marker is "the room"?
        var observer = scan.cameraPoseAtDetection.position;
        if (observer == Vector3.zero && m_Camera != null)
            observer = m_Camera.transform.position;
        var toCamera = observer - m_Origin.position;
        var normal = m_Origin.rotation * Vector3.up;
        m_IntoRoomSign = Vector3.Dot(normal, toCamera) < 0f ? -1f : 1f;
        Debug.Log($"[ChurchOriginTest] into-room sign = {m_IntoRoomSign:+0;-0}");

        // The three coloured bars are gone.

        // Nothing is drawn per checkpoint any more.

        // The pre-rendered panel quads are gone.

        // Hand the stations to the UI in the same frame the cubes use, so a
        // card can never disagree with the marker it is standing next to.
        m_ToSession = ChurchToMarker;
        BindWorld();

        // See the class comment. This is what makes it a SLAM test.
        m_Images.enabled = false;

        m_Status = "Origin set. Walk the route.";
    }
```

Lines 841–905 of ChurchOriginTest.cs

```
    void ReadLaunchUrl(string url)
    {
        if (string.IsNullOrEmpty(url))
            return;

        Debug.Log($"[ChurchOriginTest] launched with: {url}");

        const string key = "church=";
        var at = url.IndexOf(key, System.StringComparison.Ordinal);
        if (at < 0)
            return;

        var value = url.Substring(at + key.Length);
        var amp = value.IndexOf('&');
        if (amp >= 0)
            value = value.Substring(0, amp);

        var previous = m_LaunchedForChurch;
        m_LaunchedForChurch = System.Uri.UnescapeDataString(value);

        // Remember it.
        PlayerPrefs.SetString(k_LastParishKey, m_LaunchedForChurch);
        PlayerPrefs.Save();
        Debug.Log($"[ChurchOriginTest] church = {m_LaunchedForChurch}");

        if (previous == m_LaunchedForChurch)
            return;

        // Everything derived from the parish has to go.
        m_Stations = null;
        m_Surveyed = null;
```

```
    m_Recording = StationRecording.Load(ParishId);

    // The marker library is searched for this parish's sheet, and the
    // map code names its building, so both follow from the same change.
    if (m_Vps != null)
        m_Vps.Bind(FindAnyObjectByType<ARCameraManager>(), m_Camera, MapCodeForParish);

    Debug.Log($"[ChurchOriginTest] rebound to {ParishId}: " +
              $"map {MapCodeForParish}, {TableForParish.Length} stations");

    // If an origin was already established under the WRONG parish, it is meaningless now: it placed another…
    if (m_OriginSet)
    {
        m_OriginSet = false;
        m_FromVps = false;
        m_FromFloor = false;
        if (m_Images != null) m_Images.enabled = true;
        if (m_Museum != null) m_Museum.SetVisible(false);
        m_Status = "Parish changed - scan again";
    }
}
```

SRC-27 MuseumStation.cs

File: SanctiWalkAR/Assets/Scripts/MuseumStation.cs • whole file • 242 lines in full

What this code does: What one station is.

```
using System;
using UnityEngine;

// <summary> One stop on the tour: where it is, what it is, and what to say about it.
public enum MuseumStationKind
{
    History,
    Image,
    Model,
}

[Serializable]
public class MuseumStation
{
    public string stationId;

    // <summary>Card, picture, or object. See MuseumStationKind.</summary>
    public MuseumStationKind kind = MuseumStationKind.History;
    public string stationName;

    // <summary>The small grey line under the title, e.g. "Parish Patroness".</summary>
    public string subtitle;

    // <summary>Metres from the marker. X across, Y up, Z into the room.</summary>
    public Vector3 position;

    // <summary> Which way this station's card faces, in degrees about the frame's up axis.
    public float yawDegrees;

    // <summary>How close the pilgrim must be before the bubble appears.</summary>
    public float triggerRadius = 2.5f;

    // <summary>Texture name under Resources/, or empty for no artwork.</summary>
    public string image;

    // <summary> A 3D object to stand at this station, by name under Assets/Resources/Models.
    public string model;

    // <summary> Multiplies the model's own size.
    public float modelScale = 1f;

    // <summary> Where the model stands relative to the station, in metres, in the station's own frame: x across, y…
    public Vector3 modelOffset = Vector3.zero;

    // <summary>One or two sentences, shown on the bubble.</summary>
    public string shortDescription;

    // <summary>The full text, shown on the museum card.</summary>
    public string detailedDescription;

    // ---- the same three, in both languages -------------------------- The parish wrote its history in Tagalog and…
    public string nameTl, nameEn;
    public string shortTl, shortEn;
    public string detailedTl, detailedEn;

    // <summary>Station name in the language the visitor chose.</summary>
```

```
    public string Name => AppLanguage.Pick(nameTl, nameEn, stationName);

    // <summary>Bubble line in the language the visitor chose.</summary>
    public string Short => AppLanguage.Pick(shortTl, shortEn, shortDescription);

    // <summary>Card text, and what the phone reads aloud.</summary>
    public string Detailed => AppLanguage.Pick(detailedTl, detailedEn, detailedDescription);

    // <summary> What the speaker button should say.
    public string Narration
    {
        get
        {
            var d = Detailed;
            if (!string.IsNullOrEmpty(d)) return d;
            var s2 = Short;
            if (!string.IsNullOrEmpty(s2)) return s2;
            return Name;
        }
    }

    // The three metadata rows on the card.
    public string yearInstalled;
    public string artist;
    public string material;

    // <summary>stationId of the next stop, or empty if this is the last.</summary>
    public string nextStationId;

    // ---- content still to be supplied ------------------------------ Left blank on purpose.

    // <summary>Narration clip name under Resources/, or empty for none.</summary>
    public string audioClip;

    // <summary>What kind of thing this is: statue, window, altar, artifact.</summary>
    public string poiType;

    // <summary>Longer background for the museum card, beyond the description.</summary>
    public string historyNotes;

    // <summary> A copy, so a caller can change one field without changing the shared table.
    public MuseumStation Copy() => (MuseumStation)MemberwiseClone();
}

// <summary> The stations for a given building.
public static partial class MuseumStations
{
    public static readonly MuseumStation[] Sala =
    {
        new MuseumStation
        {
            stationId = "sala-sofa",
            stationName = "Pagkakatatag, 1956",
            subtitle = "3 ng 4",
            position = new Vector3(-0.700f, -0.862f, 3.072f),
            triggerRadius = 2.0f,
            image = "station-sofa",
            shortDescription = "Pindutin upang basahin",
            detailedDescription = "",
            nextStationId = "sala-alak",
        },
        new MuseumStation
        {
            stationId = "sala-santo-nino",
```

```
            stationName = "Ang Kapilya, 1951",
            subtitle = "1 ng 4",
            position = new Vector3(0.605f, -0.583f, 1.202f),
            triggerRadius = 2.0f,
            image = "station-santo-nino",
            // Converted from the supplied GLB by Tools/glb-to-obj.py, which also moved it: the file had its base 20.6 m…
            model = "st-cyril",
            modelScale = 1f,
            // 0.7 m to one side, so it stands NEXT to the Santo Nino rather
            // than inside it. Life-size already - 0.84 x 2.08 x 0.86 m.
            modelOffset = new Vector3(0.7f, 0f, 0f),
            shortDescription = "Pindutin upang basahin",
            detailedDescription = "",
            nextStationId = "sala-tv",
        },
        new MuseumStation
        {
            stationId = "sala-alak",
            stationName = "Ang Parokya, 1956",
            subtitle = "4 ng 4",
            position = new Vector3(-0.041f, -0.357f, 5.472f),
            triggerRadius = 2.0f,
            image = "station-shelf",
            shortDescription = "Pindutin upang basahin",
            detailedDescription = "",
            nextStationId = "",
        },
        new MuseumStation
        {
            stationId = "sala-tv",
            stationName = "Pd. Albert Klaeser",
            subtitle = "2 ng 4",
            position = new Vector3(-2.761f, -0.587f, 4.187f),
            triggerRadius = 2.0f,
            image = "station-tv",
            shortDescription = "Pindutin upang basahin",
            detailedDescription = "",
            nextStationId = "sala-sofa",
        },
    };
}
```

SRC-28 ArFlowUI.cs

File: SanctiWalkAR/Assets/Scripts/ArFlowUI.cs • extract • 1423 lines in full

What this code does: The welcome, and reading a station aloud as the visitor arrives.

Lines 631–641 of ArFlowUI.cs

```
void Greet()
{
    if (m_Greeted) return;
    if (m_Tutorial != null && m_Tutorial.activeSelf) return;
    if (m_Screen != Screen.Explore) return;

    m_Greeted = true;
    m_GreetingSpeaking = true;
    DeviceSpeech.Speak(WelcomeLine(), AppLanguage.Current);
    SetSpeakerPlaying(true);
}
```

Lines 1139–1181 of ArFlowUI.cs

```
void AutoNarrate()
{
    // Not over the tutorial: the visitor is reading, and the phone
    // talking over it would be two instructions at once.
    if (m_Tutorial != null && m_Tutorial.activeSelf) return;

    // Nor over the welcome. See m_GreetingSpeaking.
    if (m_GreetingSpeaking)
    {
        // Pending as well as speaking.
        if (DeviceSpeech.IsSpeaking() || DeviceSpeech.Pending) return;
        m_GreetingSpeaking = false;
    }

    var st = m_Host.NearestStation();
    if (st == null) return;

    if (st.stationId == m_SpokenStationId)
    {
        m_PendingStationId = null;
        return;
    }

    if (st.stationId != m_PendingStationId)
    {
        m_PendingStationId = st.stationId;
        m_PendingSince = Time.unscaledTime;
        return;
    }

    if (Time.unscaledTime - m_PendingSince < NarrationDwellSeconds) return;

    m_SpokenStationId = st.stationId;
    m_PendingStationId = null;

    m_GreetingSpeaking = false;
    DeviceSpeech.Stop();
```

```
    DeviceSpeech.Speak(st.Narration, AppLanguage.Current);
    SetSpeakerPlaying(true);
}
```

Lines 1186–1207 of ArFlowUI.cs

```
void SpeakCurrent()
{
    if (DeviceSpeech.IsSpeaking())
    {
        DeviceSpeech.Stop();
        SetSpeakerPlaying(false);
        // Silencing it counts as having heard this one, or AutoNarrate
        // would start it again on the very next frame.
        var here = m_Host.NearestStation();
        if (here != null) m_SpokenStationId = here.stationId;
        return;
    }

    var st = m_Host.NearestStation();
    // A deliberate tap overrules the introduction; see m_GreetingSpeaking.
    m_GreetingSpeaking = false;
    // Before any station is in range the bar carries the welcome line
    // from the mockup, so the speaker reads that instead of nothing.
    var text = st != null ? st.Narration : WelcomeLine();
    DeviceSpeech.Speak(text, AppLanguage.Current);
    SetSpeakerPlaying(true);
}
```

SRC-29 MultiSetVps.cs

File: SanctiWalkAR/Assets/Scripts/MultiSetVps.cs • extract • 501 lines in full

What this code does: Visual positioning against the parish's spatial map.

Lines 146–161 of MultiSetVps.cs

```
public void Bind(ARCameraManager cameraManager, Camera cam, string mapCode)
{
    if (m_CameraManager != null)
        m_CameraManager.frameReceived -= OnFrameReceived;

    m_CameraManager = cameraManager;
    m_Camera = cam;
    m_MapCode = mapCode;

    // Subscribed purely so the diagnostic can say whether the image and
    // the camera transform came from the same frame.
    if (m_CameraManager != null)
        m_CameraManager.frameReceived += OnFrameReceived;

    LoadCredentials();
}
```

Lines 207–211 of MultiSetVps.cs

```
public void Localize()
{
    if (Busy) return;
    StartCoroutine(Run());
}
```

11. Build, Packaging and Deployment

SRC-30 capacitor.config.ts

File: SanctDemo/capacitor.config.ts • whole file • 69 lines in full

What this code does: Packaging the web application for Android.

```
import type { CapacitorConfig } from '@capacitor/cli'

// Packages the SanctiWalk web app as a real Android application.
const LAN_TEST = process.env.CAP_LAN_TEST === '1'

const config: CapacitorConfig = {
  appId: 'ph.edu.sti.sanctiwalk',
  appName: 'SanctiWalk',
  // `npx vite build` writes here.
  webDir: 'dist',

  android: {
    // The scanner and the AR tour both need the camera, and a WebView only grants getUserMedia on a secure origin.
    allowMixedContent: LAN_TEST,
  },

  plugins: {
    FirebaseAuthentication: {
      // Only the Google account picker is wanted from the native side.
      skipNativeAuth: true,
      // Without this the Google SDK is not loaded on Android at all and the picker never opens.
      providers: ['google.com'],
    },
  },

  server: {
    androidScheme: 'https',
    // Android has refused cleartext traffic by default since API 28, on top of the mixed-content rule above.
    cleartext: LAN_TEST,
  },
}

export default config
```

SRC-31 AndroidManifest.xml

File: SanctDemo/android/app/src/main/AndroidManifest.xml • whole file • 95 lines in full

What this code does: Declared permissions.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">

        <!-- Portrait only. Every screen here is laid out as a single phone
             column - the parish band, the bulletin rail, the rosary - and
             rotated there is no second column for the extra width to become,
             so landscape only made each screen shorter. That broke the
             modals outright: the rosary settings card is about 530dp tall,
             which in a 360dp-high landscape window overflowed by 84dp at
             each end and took its own close button off the top of the
             screen with nothing left to scroll. The modals are height-safe
             now as well, because this lock does not apply to the same app
             opened in a browser. -->
        <activity
            android:screenOrientation="portrait"
            android:configChanges="orientation|keyboardHidden|keyboard|screenSize|locale|smallestScreenSize|screenLayout|uiMode|navigation|density"
            android:name=".MainActivity"
            android:label="@string/title_activity_main"
            android:theme="@style/AppTheme.NoActionBarLaunch"
            android:launchMode="singleTask"
            android:exported="true">

            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>

        </activity>

        <provider
            android:name="androidx.core.content.FileProvider"
            android:authorities="${applicationId}.fileprovider"
            android:exported="false"
            android:grantUriPermissions="true">
            <meta-data
                android:name="android.support.FILE_PROVIDER_PATHS"
                android:resource="@xml/file_paths"></meta-data>
        </provider>
    </application>

    <!-- Permissions -->

    <uses-permission android:name="android.permission.INTERNET" />

    <!-- The Scan tab opens the camera for AI recognition and QR codes. A
         WebView only grants getUserMedia when the app also holds the OS
         permission, so declaring INTERNET alone is not enough. -->
    <uses-permission android:name="android.permission.CAMERA" />
```

```
    <!-- The Rosary's glow responds to the voice of whoever is praying, which
         it measures with a loudness meter over the microphone
         (public/rosary/index.html, startMeter). Leaving these undeclared did
         not merely disable that: Capacitor maps an AUDIO_CAPTURE request from
         the WebView onto RECORD_AUDIO and launches it, the framework refuses
         an undeclared permission outright, and the result came back to
         BridgeWebChromeClient twice - the second call to deny() threw
         IllegalStateException on the main thread and took the whole app down.
         That was the Rosary "crashing": a FATAL EXCEPTION, not a slow page.
         MODIFY_AUDIO_SETTINGS is requested alongside RECORD_AUDIO by that
         same Capacitor code path, so both have to be here. -->
    <uses-permission android:name="android.permission.RECORD_AUDIO" />
    <uses-permission android:name="android.permission.MODIFY_AUDIO_SETTINGS" />

    <!-- Mass reminders, and the "tell me when this opens" bells on the
         schedule, ministries and sacraments.

         Android 13 made posting a notification a runtime permission. The
         app already asked for it through the Capacitor plugin and the
         answer was always no - silently, with no system dialog, because
         a permission the manifest does not declare cannot be granted and
         requestPermissions() just returns denied. So notifications were
         the one thing the app never prompted for on install, and every
         Mass change notice went nowhere. It has to be declared here for
         the prompt to exist at all. -->
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

    <!-- Parish detection and walking directions. Coarse is listed too so the
         app still installs and degrades gracefully on a device that refuses
         precise location. -->
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

    <!-- Optional, deliberately. Marking the camera required would stop the
         app installing on any device without one, and every camera feature
         already has a working fallback (manual station picking). -->
    <uses-feature android:name="android.hardware.camera" android:required="false" />
</manifest>
```

SRC-32 vercel.json

File: SanctDemo/vercel.json • whole file • 43 lines in full

What this code does: Web hosting and headers.

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "installCommand": "npm install --ignore-scripts",
  "buildCommand": "npm run build:app",
  "outputDirectory": "dist",
  "framework": "vite",
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ],
  "headers": [
    {
      "source": "/(.*)",
      "headers": [
        { "key": "X-Content-Type-Options", "value": "nosniff" },
        { "key": "Referrer-Policy", "value": "strict-origin-when-cross-origin" },
        { "key": "X-Frame-Options", "value": "SAMEORIGIN" }
      ]
    },
    {
      "source": "/assets/(.*)",
      "headers": [
        { "key": "Cache-Control", "value": "public, max-age=31536000, immutable" }
      ]
    },
    {
      "source": "/sw.js",
      "headers": [
        { "key": "Cache-Control", "value": "public, max-age=0, must-revalidate" },
        { "key": "Service-Worker-Allowed", "value": "/" }
      ]
    },
    {
      "source": "/manifest.webmanifest",
      "headers": [
        { "key": "Content-Type", "value": "application/manifest+json" },
        { "key": "Cache-Control", "value": "public, max-age=0, must-revalidate" }
      ]
    }
  ]
}
```

Index of Listings

ID	File	Path	Lines in file	Lines shown
SRC-01	main.tsx	SanctDemo/src/main.tsx	120	91
SRC-02	RootGate.tsx	SanctDemo/src/components/RootGate.tsx	70	19
SRC-03	sundayMasses.ts	SanctDemo/src/lib/sundayMasses.ts	199	33
SRC-04	schedule.ts	SanctDemo/src/lib/schedule.ts	177	17
SRC-05	massScheduleWatch.ts	SanctDemo/src/lib/massScheduleWatch.ts	243	50
SRC-06	openingSubscriptions.ts	SanctDemo/src/lib/openingSubscriptions.ts	118	13
SRC-07	speech.ts	SanctDemo/src/lib/speech.ts	227	71
SRC-08	DailyRosary.tsx	SanctDemo/src/components/DailyRosary.tsx	203	92
SRC-09	sanctiGrounding.ts	SanctDemo/src/lib/sanctiGrounding.ts	244	69
SRC-10	askSancti.ts	SanctDemo/src/lib/askSancti.ts	84	35
SRC-11	scanQuota.ts	SanctDemo/src/lib/scanQuota.ts	104	14
SRC-12	followCamera.ts	SanctDemo/src/lib/followCamera.ts	190	25
SRC-13	geo.ts	SanctDemo/src/lib/geo.ts	42	28
SRC-14	server.ts	SanctDemo/server.ts	1080	46
SRC-15	server.ts	SanctDemo/server.ts	1080	248
SRC-16	server.ts	SanctDemo/server.ts	1080	99
SRC-17	firebase.ts	SanctDemo/src/lib/firebase.ts	11	11
SRC-18	authFlow.ts	SanctDemo/src/lib/authFlow.ts	262	18
SRC-19	availability.ts	SanctDemo/src/lib/availability.ts	102	19
SRC-20	firestore.rules	SanctDemo/firestore.rules	409	258
SRC-21	AdminItems.tsx	SanctDemo/src/admin/AdminItems.tsx	201	121
SRC-22	pageEvents.ts	SanctDemo/src/lib/pageEvents.ts	185	41
SRC-23	deploy-stations.py	SanctiWalkAR/Tools/deploy-stations.py	302	59
SRC-24	save_server.py	SanctiWalkAR/Tools/viewer/save_server.py	77	68
SRC-25	MarkerScan.cs	SanctiWalkAR/Assets/Scripts/MarkerScan.cs	83	48
SRC-26	ChurchOriginTest.cs	SanctiWalkAR/Assets/Scripts/ChurchOriginTest.cs	2602	178
SRC-27	MuseumStation.cs	SanctiWalkAR/Assets/Scripts/MuseumStation.cs	242	161
SRC-28	ArFlowUI.cs	SanctiWalkAR/Assets/Scripts/ArFlowUI.cs	1423	73
SRC-29	MultiSetVps.cs	SanctiWalkAR/Assets/Scripts/MultiSetVps.cs	501	21
SRC-30	capacitor.config.ts	SanctDemo/capacitor.config.ts	69	33
SRC-31	AndroidManifest.xml	SanctDemo/android/app/src/main/AndroidManifest.xml	95	95
SRC-32	vercel.json	SanctDemo/vercel.json	43	43